循序渐进，学习开发 xv6-riscv
第 19 章 进程和堆

汪辰

目录

01

堆（Heap）

堆（heap）的概念

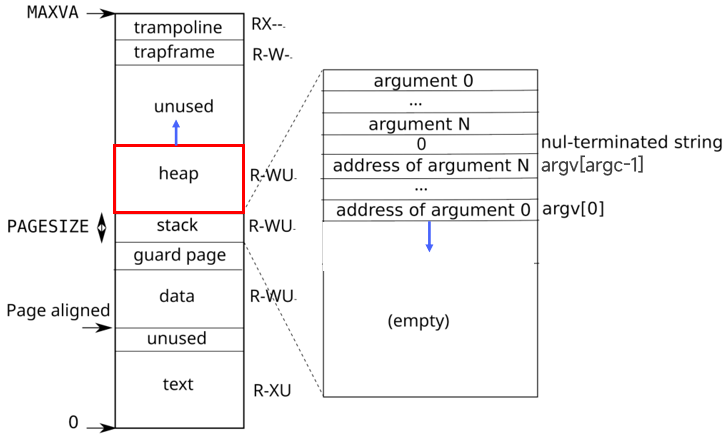


Figure 3.4: A process's user address space, with its initial stack.

break

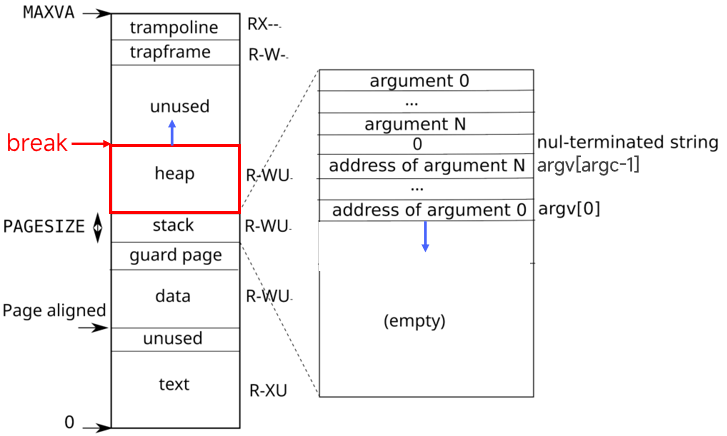


Figure 3.4: A process's user address space, with its initial stack.

02

系统调用 sbrk

系统调用 sbrk

```
brk(2)                          System Calls Manual                          brk(2)

NAME
       brk, sbrk - change data segment size

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int brk(void *addr);
       void *sbrk(intptr_t increment);
```

man 2 sbrk

user/user.h

char* sbrk(int increment);

```
diff --git a/kernel/syscall.c b/kernel/syscall.c
index 8955dca..c997e3a 100644
--- a/kernel/syscall.c
+++ b/kernel/syscall.c
@@ -87,6 +87,7 @@ extern uint64 sys_read(void);
 extern uint64 sys_kill(void);
 extern uint64 sys_exec(void);
 extern uint64 sys_getpid(void);
+extern uint64 sys_sbrk(void);
 extern uint64 sys_pause(void);
 extern uint64 sys_uptime(void);
 extern uint64 sys_write(void);
@@ -101,6 +102,7 @@ static uint64 (*syscalls[])(void) = {
 [SYS_kill]    sys_kill,
 [SYS_exec]    sys_exec,
 [SYS_getpid]  sys_getpid,
+[SYS_sbrk]    sys_sbrk,
 [SYS_pause]   sys_pause,
 [SYS_uptime]  sys_uptime,
 [SYS_write]   sys_write,
```

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index ccc722f..e154cee 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
@@ -35,6 +35,19 @@ sys_wait(void)
   return kwait(p);
 }

+uint64
+sys_sbrk(void)
+{
+  uint64 addr;
+  int n;
+
+  argint(0, &n);
+  addr = myproc()->sz;
+  if(growproc(n) < 0)
+    return -1;
+  return addr;
+}
```

系统调用 sbrk

```c
uint64 sys_sbrk(void)
{
  uint64 addr;
  int n;

  argint(0, &n);
  addr = myproc()->sz;
  if(growproc(n) < 0)
    return -1;
  return addr;
}
```

```c
int growproc(int n)
{
  uint64 sz;
  struct proc *p = myproc();

  sz = p->sz;
  if(n > 0){
    if(sz + n > TRAPFRAME) {
      return -1;
    }
    if((sz = uvmalloc(p->pagetable,
                      sz, sz + n,
                      PTE_W)) == 0) {
      return -1;
    }
  } else if(n < 0){
    sz = uvmdealloc(p->pagetable, sz, sz + n);
  }
  p->sz = sz;
  return 0;
}
```

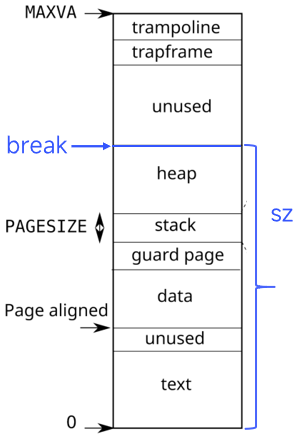


Figure 3.4: A process's user address space, wit

系统调用 sbrk

```c
uint64 sys_sbrk(void)
{
  uint64 addr;
  int n;

  argint(0, &n);
  addr = myproc()->sz;
  if(growproc(n) < 0)
    return -1;
  return addr;
}
```

```c
int growproc(int n)
{
  uint64 sz;
  struct proc *p = myproc();

  sz = p->sz;
  if(n > 0){
    if(sz + n > TRAPFRAME) {
      return -1;
    }
    if((sz = uvmalloc(p->pagetable,
                      sz, sz + n,
                      PTE_W)) == 0) {
      return -1;
    }
  } else if(n < 0){
    sz = uvmdealloc(p->pagetable, sz, sz + n);
  }
  p->sz = sz;
  return 0;
}
```

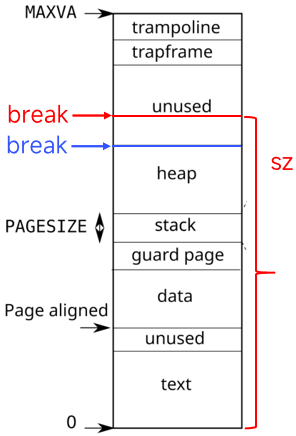
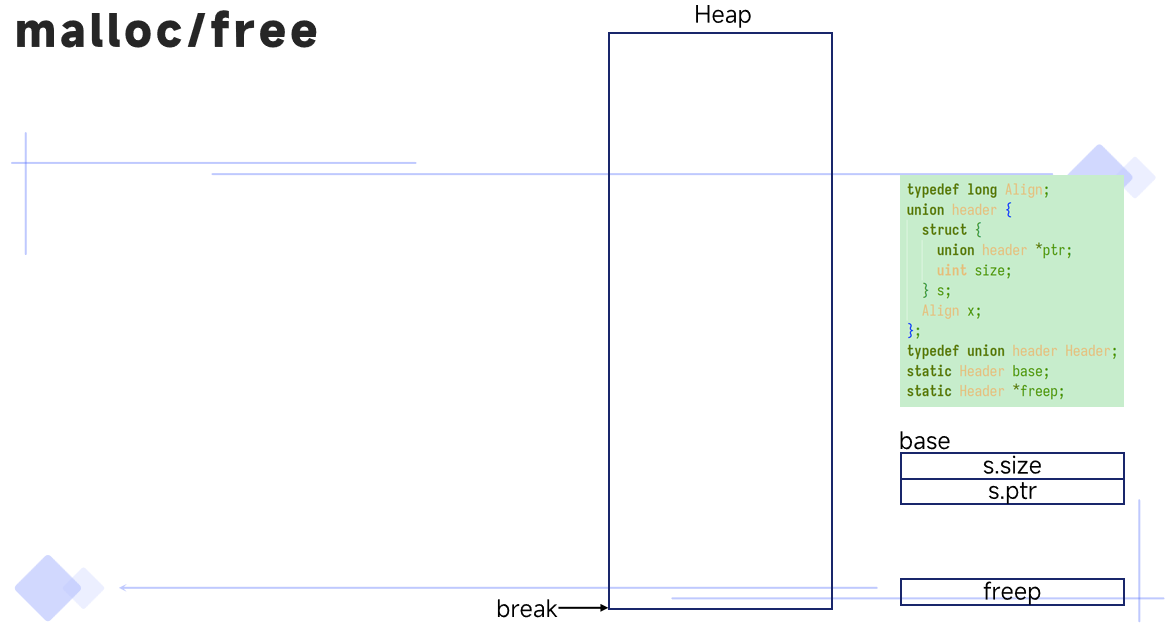


Figure 3.4: A process's user address space, wit

03

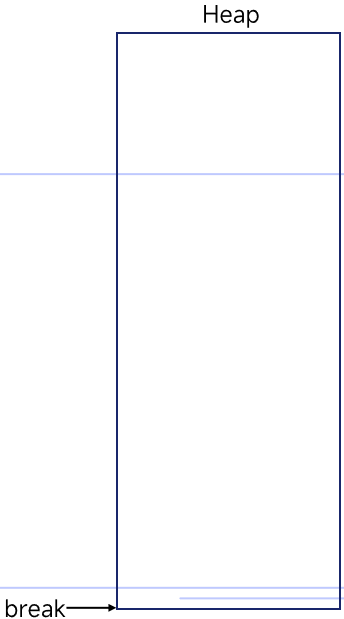
库函数 malloc/free

malloc/free


Heap
typedef long Align;
union header {
struct {
union header *ptr;
uint size;
} s;
Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
base
s.size
s.ptr
freep
break

malloc/free

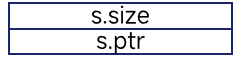
```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

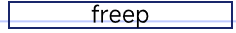



```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```

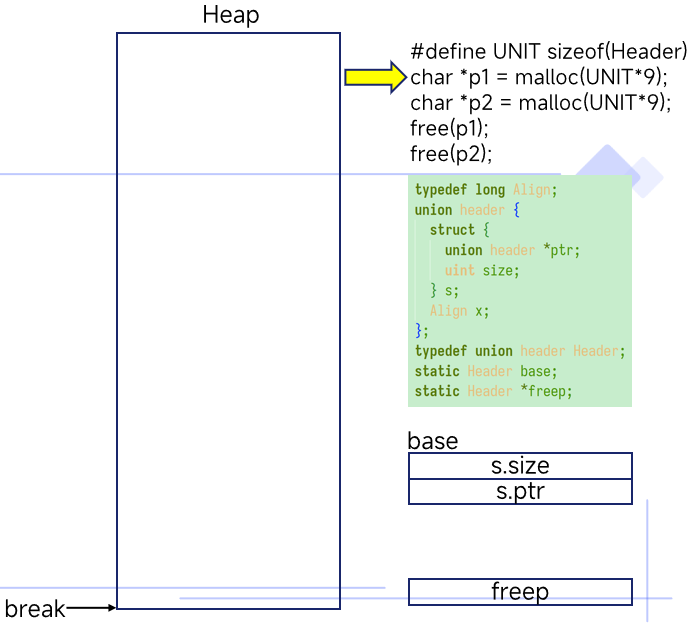
base





malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```



malloc/free

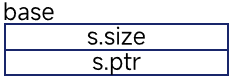
```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

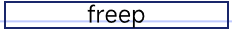
为了简化对齐操作，所有分配的内存块的大小均要求为头部大小（sizeof（Header））的整数倍；

计算 nbytes 字节需要多少个 Header 单元，需要向上取整，最后再加 1 给 Header 自己用（Header 作为 meta 数据也是分配的内存块的一部分）

```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```

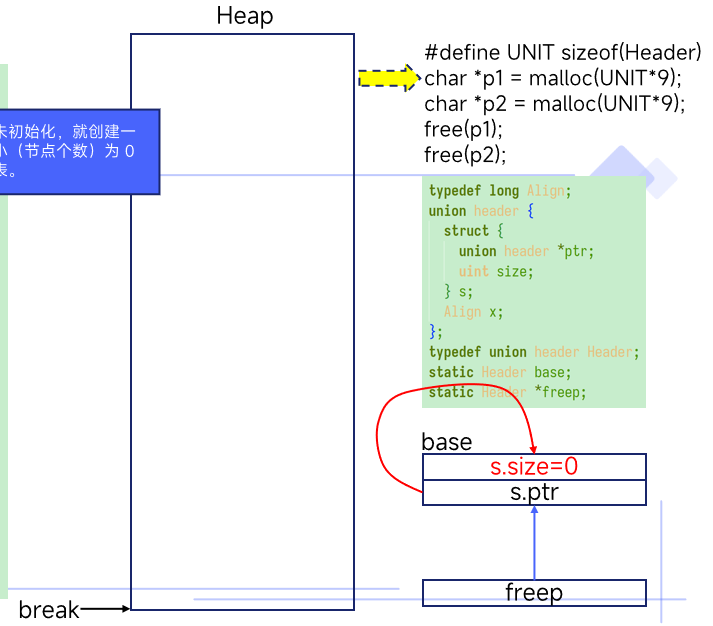




malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

如果链表未初始化，就创建一个初始大小（节点个数）为 0 的空闲链表。

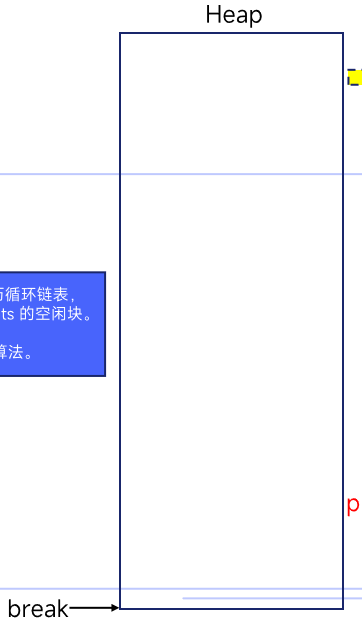


malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

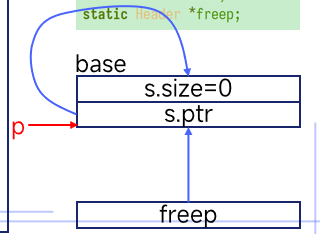
从 freep 开始，循环遍历循环链表，寻找第一个 size >= nunits 的空闲块。

“首次适应”（First Fit）算法。




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



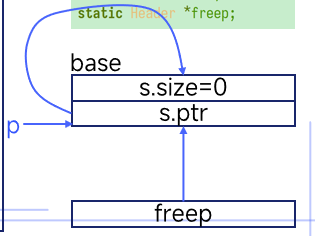
malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

如果扫描一圈都没找到合适的空闲块就调用 morecore() 向内核申请更多内存。
申请成功则返回 for 循环继续搜索;失败则返回 0。


```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```

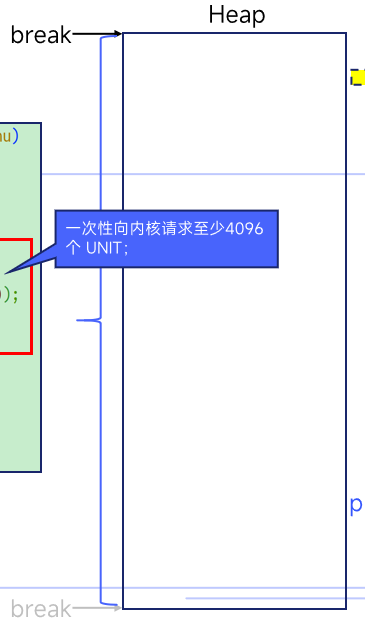


malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof
  if((prevp = freep) == 0){
    base.s.ptr = freep = pr
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; p
    if(p->s.size >= nunits)
      if(p->s.size ==
        prevp->s.ptr = p->s
      else {
        p->s.size -= nunits
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1)
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

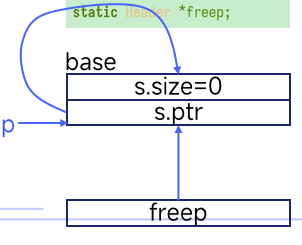
```
static Header* morecore(uint nu)
{
  char *p;
  Header *hp;

  if(nu < 4096)
    nu = 4096;
  p = sbrk(nu * sizeof(Header));
  if(p == SBRK_ERROR)
    return 0;
  hp = (Header*)p;
  hp->s.size = nu;
  free((void*)(hp + 1));
  return freep;
}
```

一次性向内核请求至少4096个 UNIT;




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```

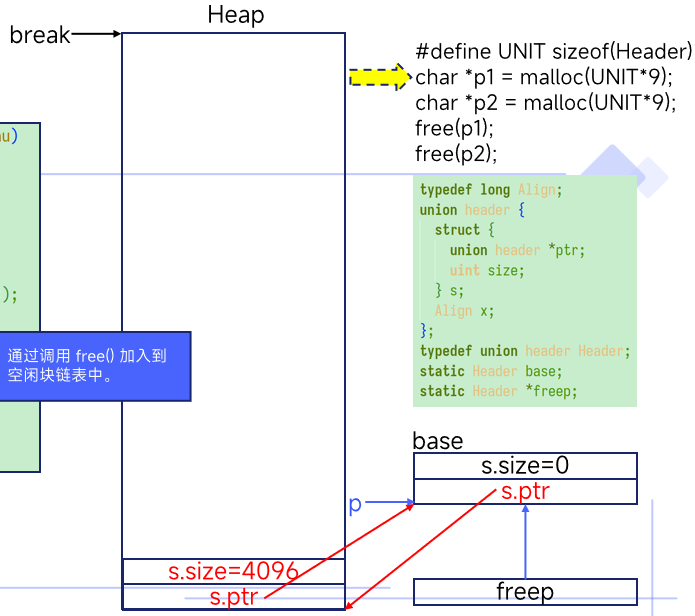


malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof
  if((prevp = freep) == 0){
    base.s.ptr = freep = pr
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; p
    if(p->s.size >= nunits)
      if(p->s.size == nunit
        prevp->s.ptr = p->s
      else {
        p->s.size -= nunits
        p += p->s.size;
        p->s.size = n
      }
      freep = prevp;
      return (void*)(p + 1)
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

```
static Header* morecore(uint nu)
{
  char *p;
  Header *hp;

  if(nu < 4096)
    nu = 4096;
  p = sbrk(nu * sizeof(Header));
  if(p == SBRK_ERROR)
    return 0;
  hp = (Header*)p;
  hp->s.size = nu;
  free((void*)(hp + 1));
  return freep;
}
```

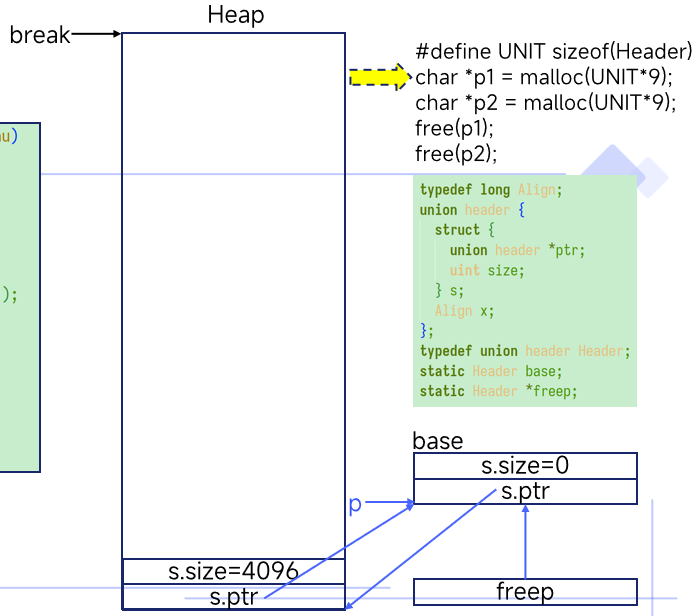
通过调用 free() 加入到空闲块链表中。



malloc/free

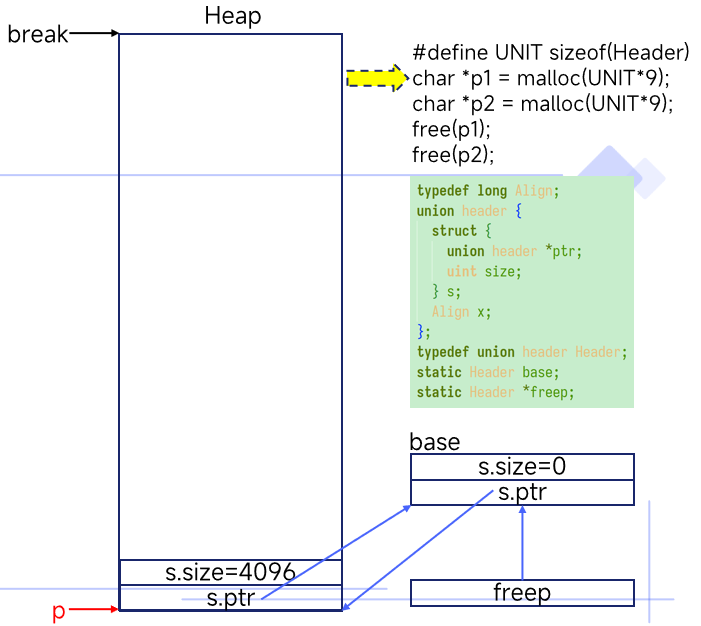
```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof
  if((prevp = freep) == 0){
    base.s.ptr = freep = pr
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; p
    if(p->s.size >= nunits)
      if(p->s.size == nunit
        prevp->s.ptr = p->s
      else {
        p->s.size -= nunits
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1)
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

```
static Header* morecore(uint nu)
{
  char *p;
  Header *hp;

  if(nu < 4096)
    nu = 4096;
  p = sbrk(nu * sizeof(Header));
  if(p == SBRK_ERROR)
    return 0;
  hp = (Header*)p;
  hp->s.size = nu;
  free((void*)(hp + 1));
  return freep;
}
```



malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

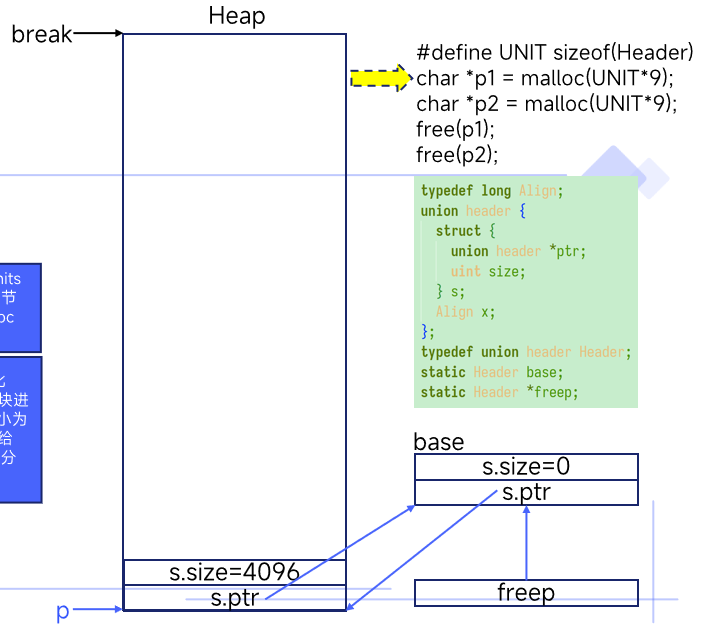


malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

如果块大小 p->s.size 和 nunits 相等，完美。 将 p 所指向的节点从链表摘出来返回给 malloc 的调用者。

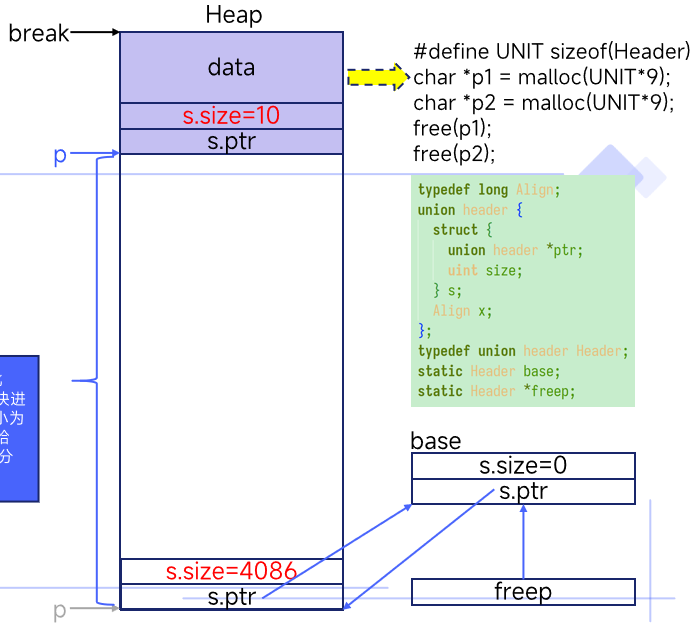
如果节点块大小 p->s.size 比 nunits 的大，则将该节点的块进行分裂，将位于高地址的大小为 nunits 的部分分裂出来返回给 malloc 的调用者；剩余的部分留在链表中。



malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

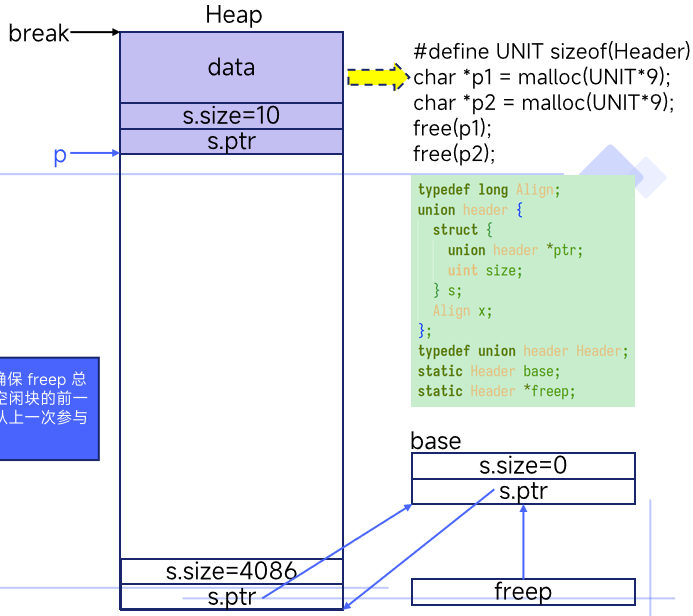
如果节点块大小 p->s.size 比 nunits 的大，则将该节点的块进行分裂，将位于高地址的大小为 nunits 的部分分裂出来返回给 malloc 的调用者；剩余的部分留在链表中。



malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

将 freep 更新为 prevp。确保 freep 总是指向上一次参与分配的空闲块的前一块，下一次搜索时会继续从上一次参与分配的空闲块开始。



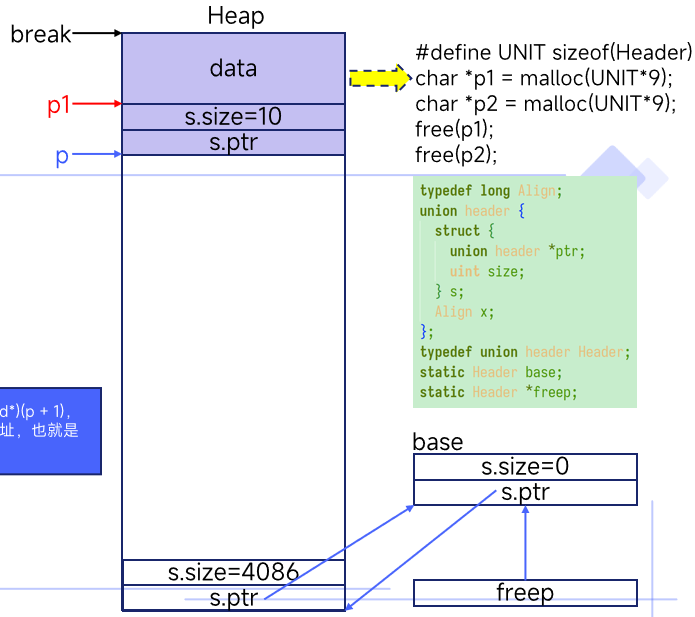

```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```

malloc/free

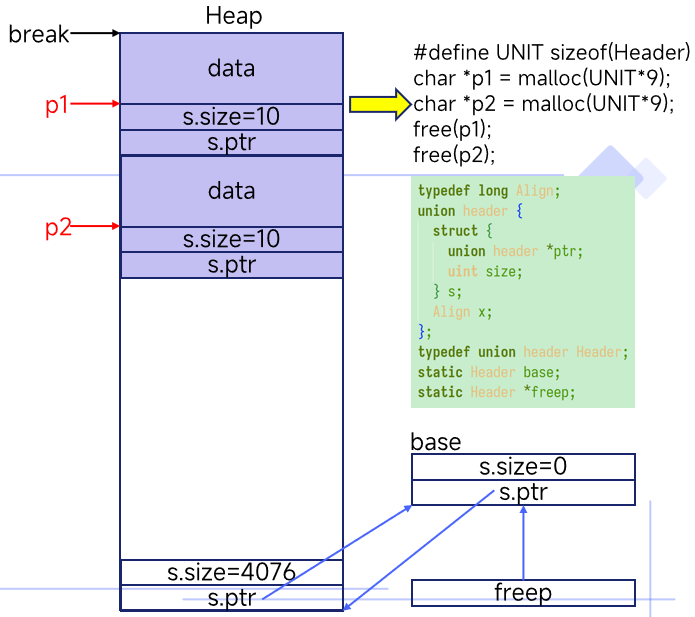
```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```

返回给用户的地址是 (void*)(p + 1)，即 Header 之后的那个地址，也就是真正的可用数据区域。



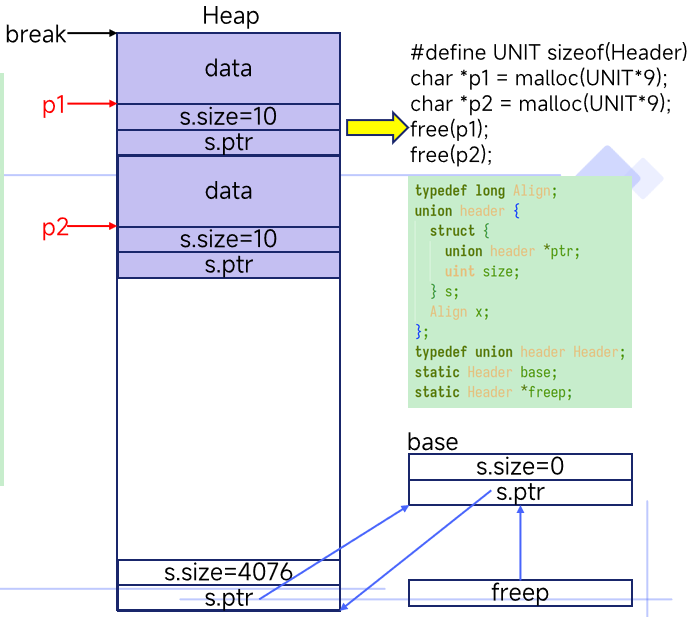
malloc/free

```
void* malloc(uint nbytes)
{
  Header *p, *prevp;
  uint nunits;
  nunits = (nbytes + sizeof(Header) - 1)/sizeof(Header) + 1;
  if((prevp = freep) == 0){
    base.s.ptr = freep = prevp = &base;
    base.s.size = 0;
  }
  for(p = prevp->s.ptr; ; prevp = p, p = p->s.ptr){
    if(p->s.size >= nunits){
      if(p->s.size == nunits)
        prevp->s.ptr = p->s.ptr;
      else {
        p->s.size -= nunits;
        p += p->s.size;
        p->s.size = nunits;
      }
      freep = prevp;
      return (void*)(p + 1);
    }
    if(p == freep)
      if((p = morecore(nunits)) == 0)
        return 0;
  }
}
```



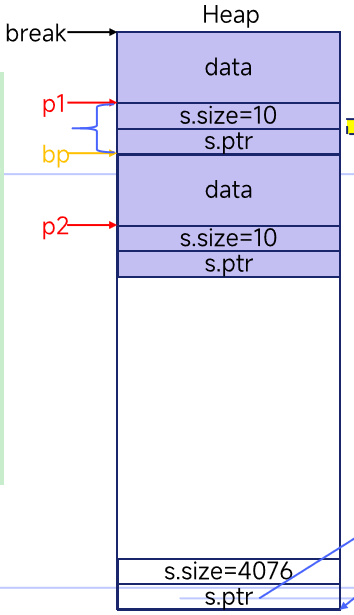
malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```



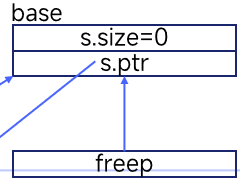
malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```

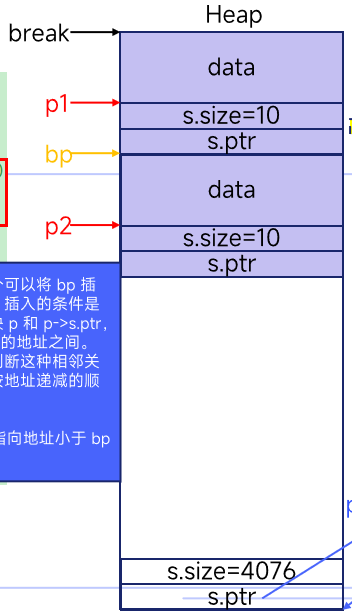


malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

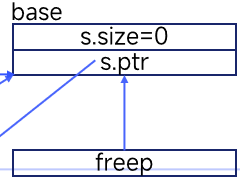
在循环链表中寻找一个可以将 bp 插入进去的合适的位置。插入的条件是存在两个相邻的空闲块 p 和 p->s.ptr，满足 bp 的地址在它们的地址之间。之所以可以根据地址判断这种相邻关系是因为空闲链表是按地址递减的顺序进行维护的。

注意：搜索结束时 p 指向地址小于 bp 的那个块。




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

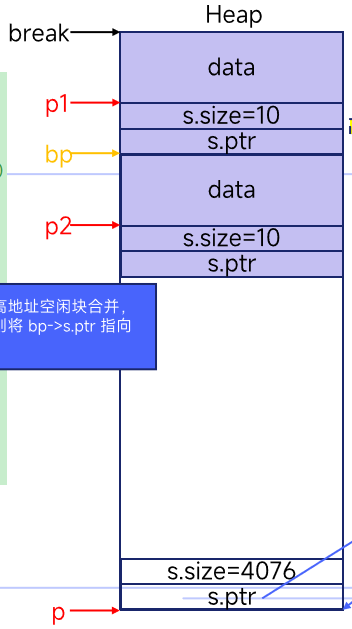
```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



malloc/free

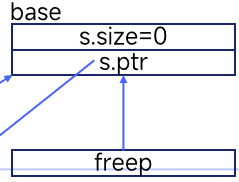
```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

先尝试和相邻的高地址空闲块合并，如果合并不了，则将 bp->s.ptr 指向 p->s.ptr。




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

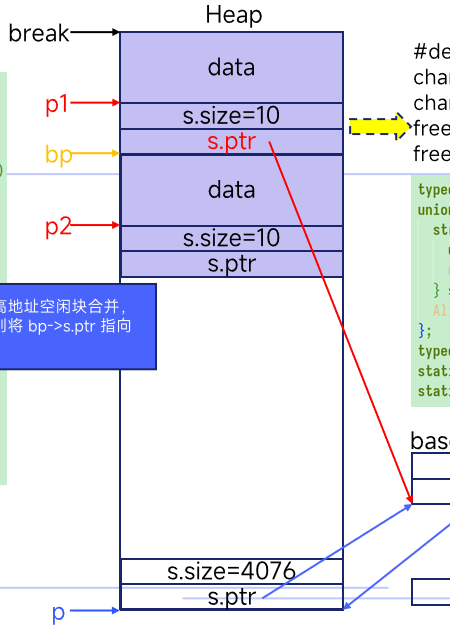
```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

先尝试和相邻的高地址空闲块合并，如果合并不了，则将 bp->s.ptr 指向 p->s.ptr。



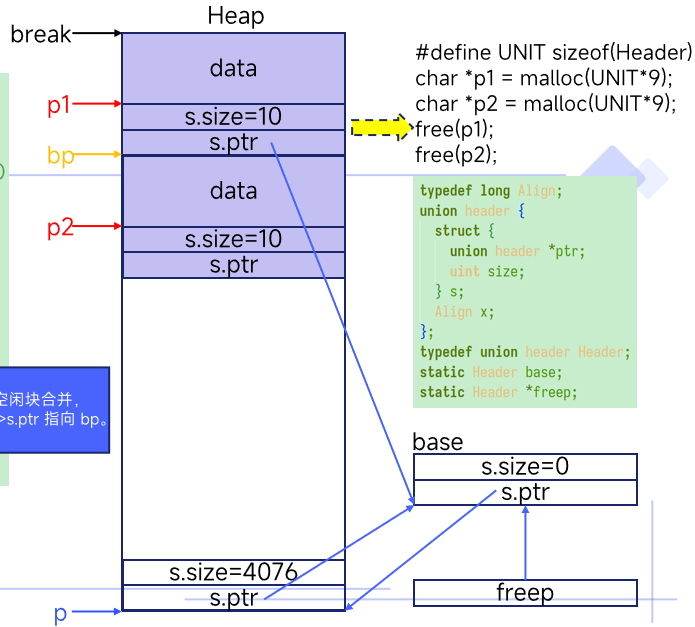

```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```

malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

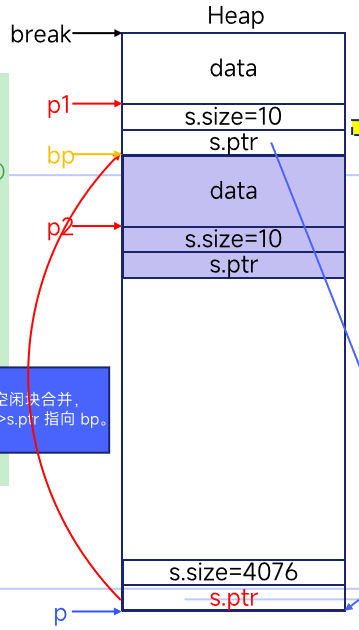
再尝试和相邻的低地址空闲块合并，如果合并不了，则将 p->s.ptr 指向 bp。



malloc/free

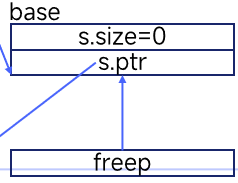
```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

再尝试和相邻的低地址空闲块合并，如果合并不了，则将 p->s.ptr 指向 bp。



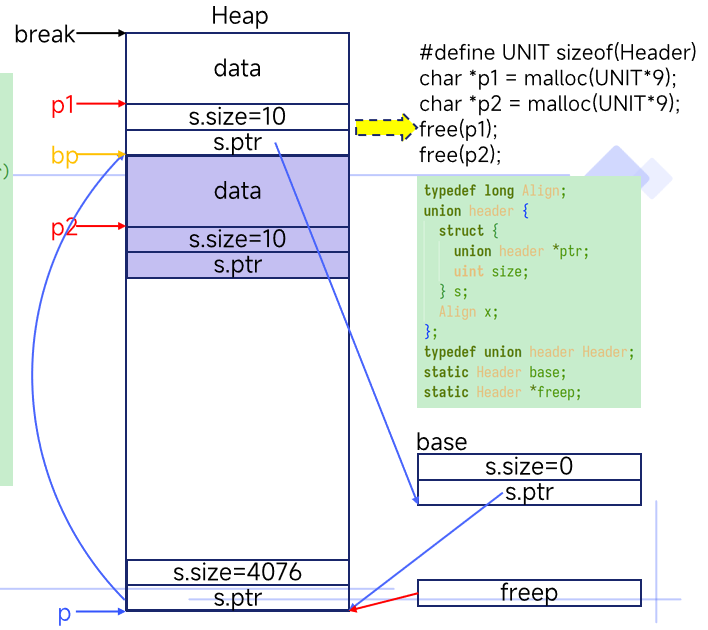

```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2= malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



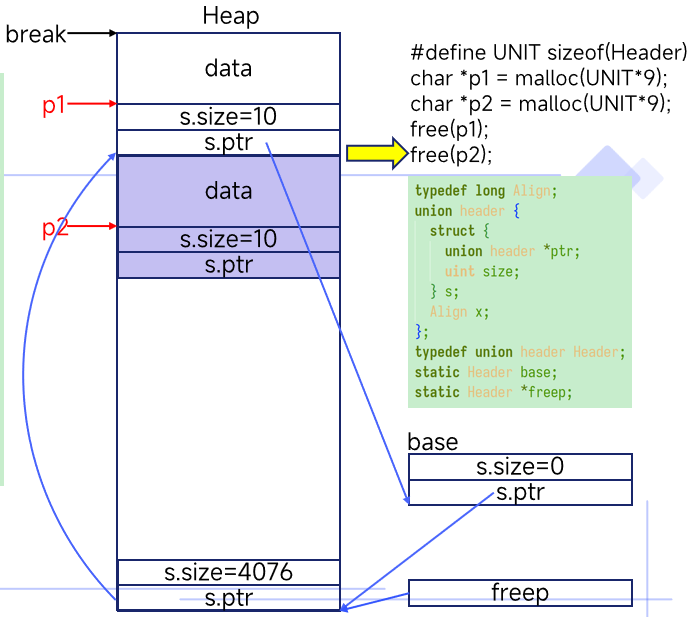
malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```



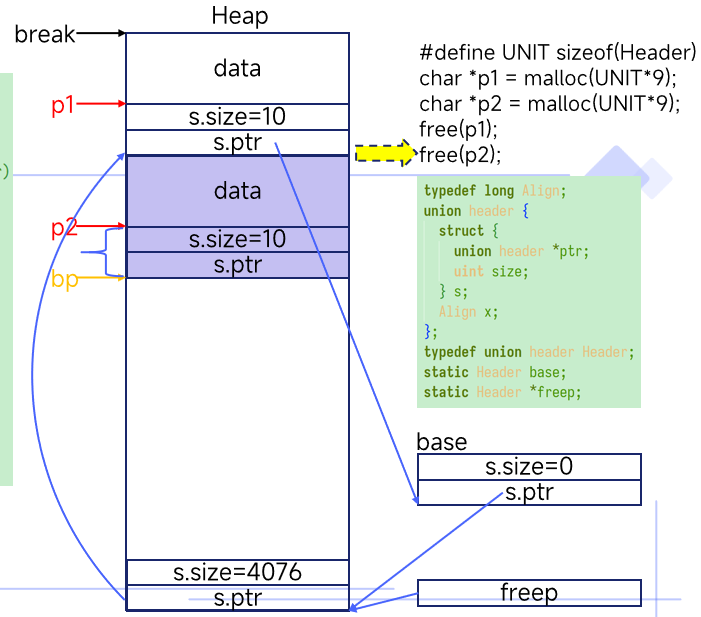
malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```



malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

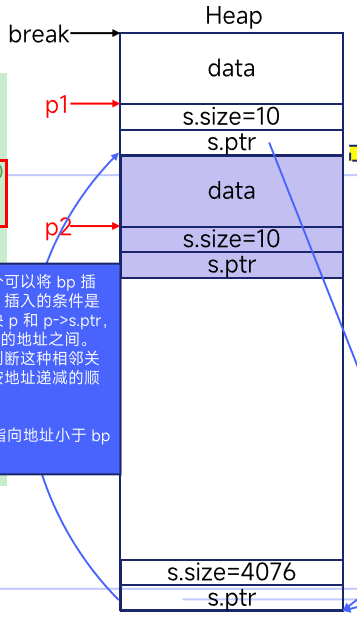


malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

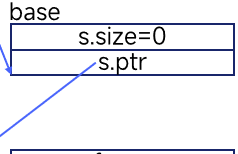
在循环链表中寻找一个可以将 bp 插入进去的合适的位置。插入的条件是存在两个相邻的空闲块 p 和 p->s.ptr，满足 bp 的地址在它们的地址之间。之所以可以根据地址判断这种相邻关系是因为空闲链表是按地址递减的顺序进行维护的。

注意：搜索结束时 p 指向地址小于 bp 的那个块。




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

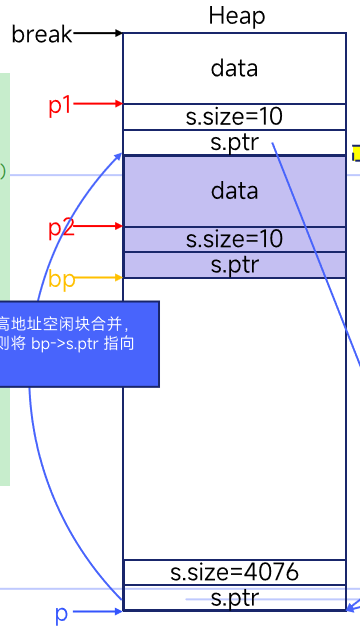
```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



malloc/free

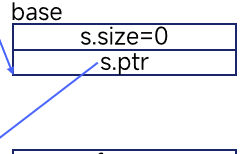
```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

先尝试和相邻的高地址空闲块合并，如果合并不了，则将 bp->s.ptr 指向 p->s.ptr。




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

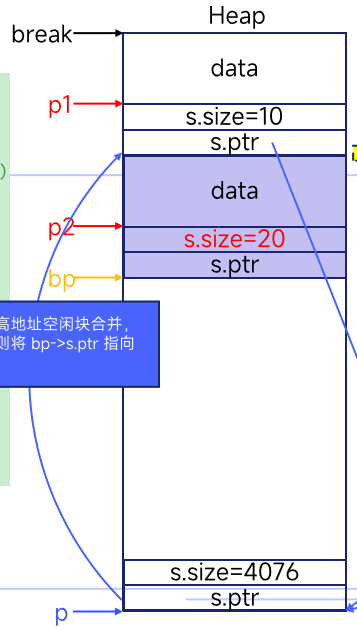
```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



malloc/free

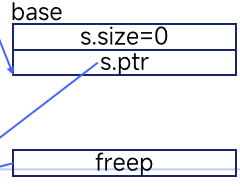
```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

先尝试和相邻的高地址空闲块合并，如果合并不了，则将 bp->s.ptr 指向 p->s.ptr。




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

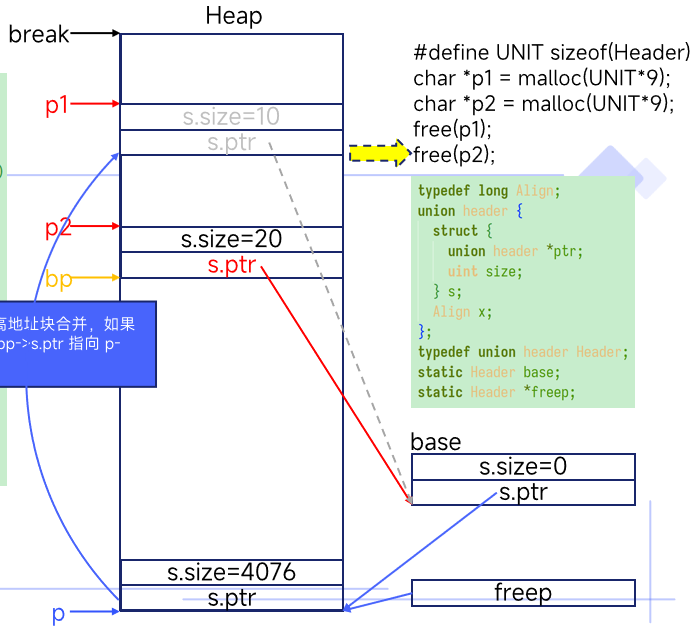
```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

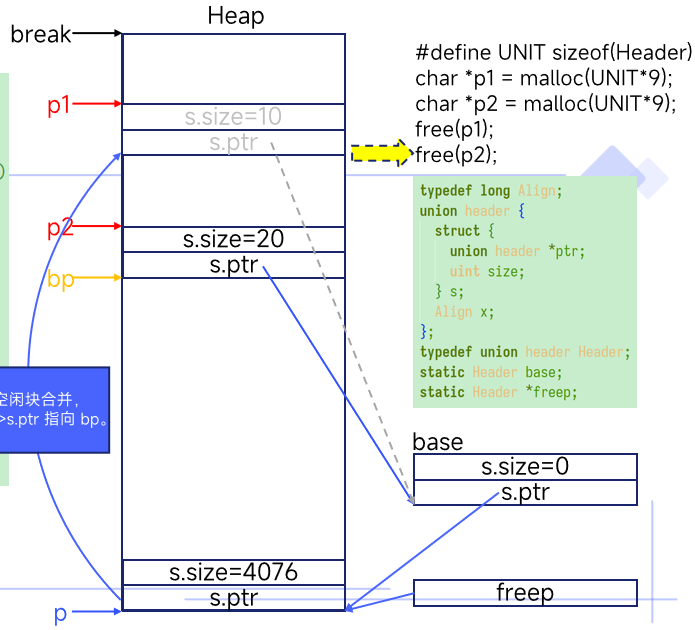
先尝试和相邻的高地址块合并，如果合并不了，则将 bp->s.ptr 指向 p->s.ptr。



malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

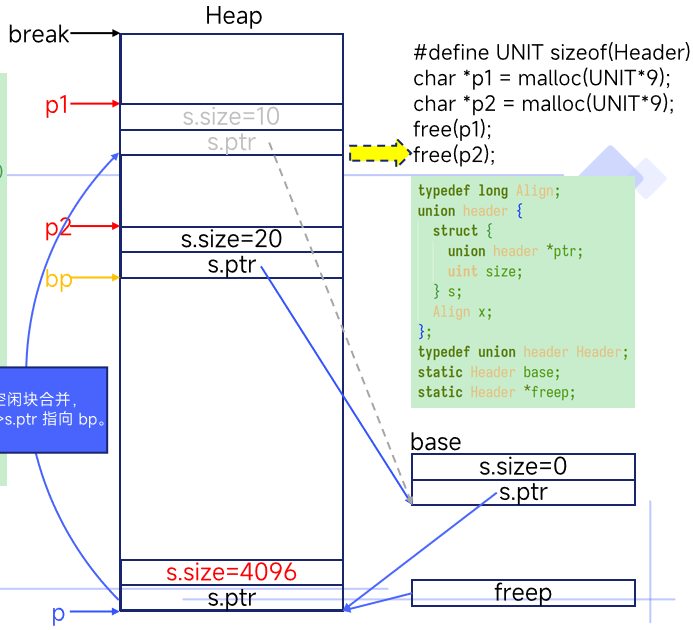
再尝试和相邻的低地址空闲块合并，如果合并不了，则将 p->s.ptr 指向 bp。



malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

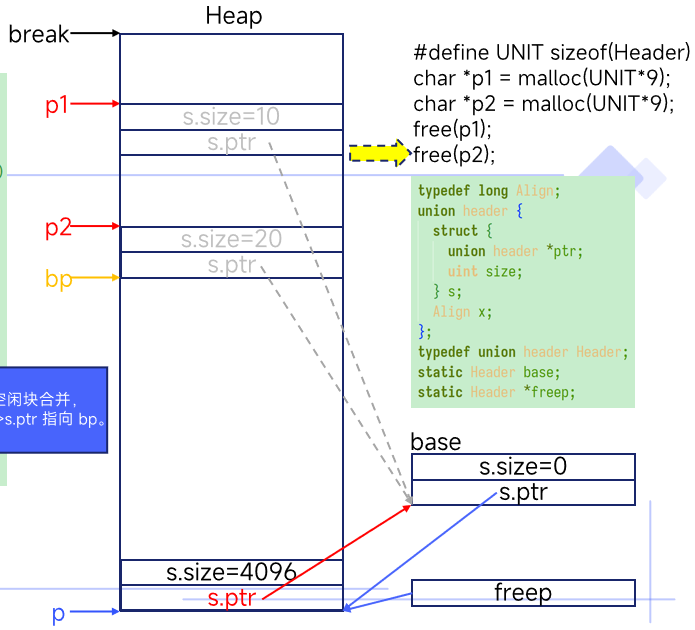
再尝试和相邻的低地址空闲块合并，如果合并不了，则将 p->s.ptr 指向 bp。



malloc/free

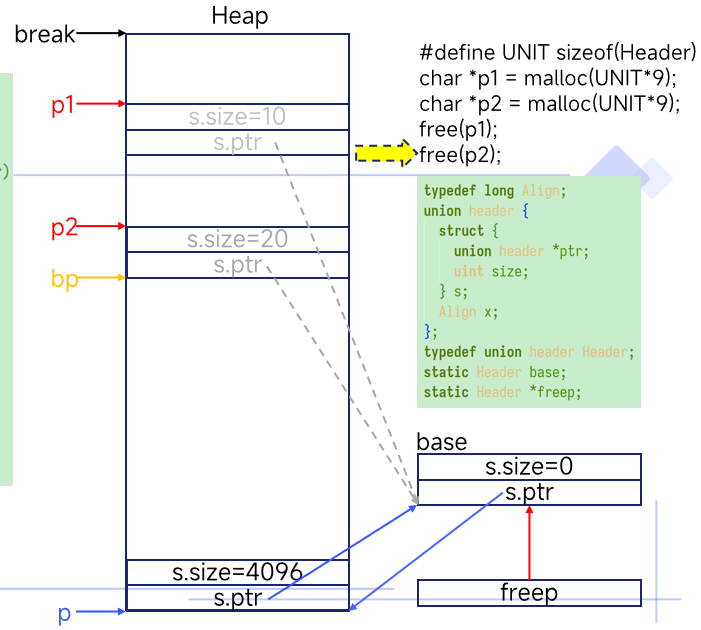
```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```

再尝试和相邻的低地址空闲块合并，如果合并不了，则将 p->s.ptr 指向 bp。



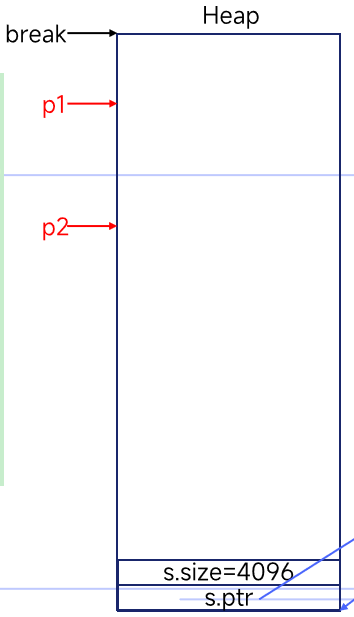
malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```



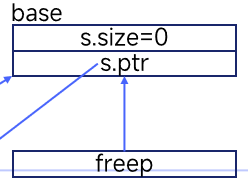
malloc/free

```
void free(void *ap)
{
  Header *bp, *p;
  bp = (Header*)ap - 1;
  for(p = freep; !(bp > p && bp < p->s.ptr); p = p->s.ptr)
    if(p >= p->s.ptr && (bp > p || bp < p->s.ptr))
      break;
  if(bp + bp->s.size == p->s.ptr){
    bp->s.size += p->s.ptr->s.size;
    bp->s.ptr = p->s.ptr->s.ptr;
  } else
    bp->s.ptr = p->s.ptr;
  if(p + p->s.size == bp){
    p->s.size += bp->s.size;
    p->s.ptr = bp->s.ptr;
  } else
    p->s.ptr = bp;
  freep = p;
}
```




```
#define UNIT sizeof(Header)
char *p1 = malloc(UNIT*9);
char *p2 = malloc(UNIT*9);
free(p1);
free(p2);
```

```
typedef long Align;
union header {
  struct {
    union header *ptr;
    uint size;
  } s;
  Align x;
};
typedef union header Header;
static Header base;
static Header *freep;
```



03

延迟分配

缺页异常（Page Fault）

当使用的虚拟地址在页表中没有被映射，或者映射的 PTE_V 标志为清除状态（0），或者指令尝试执行针对该虚拟地址禁止的操作时，RISC-V CPU 会产生“**缺页（page-fault）**”异常。

组合利用页表和缺页异常可以实现强大的功能。

- 写时复制（copy-on-write）
- 内存映射文件（memory-mapped files）
- 延迟内存分配（lazy memory allocation）

sbrk 和 sbrklazy

```
brk(2)                          System Calls Manual                          brk(2)

NAME
       brk, sbrk - change data segment size

LIBRARY
       Standard C library (libc, -lc)

SYNOPSIS
       #include <unistd.h>

       int brk(void *addr);
       void *sbrk(intptr_t increment);
```

man 2 sbrk

user/user.h

```
char* sbrk(int increment);
char* sbrklazy(int increment);
```

```
diff --git a/user/ulib.c b/user/ulib.c
index 3432772..8a9683c 100644
--- a/user/ulib.c
+++ b/user/ulib.c
@@ -60,3 +61,14 @@ gets(char *buf, int max)
   buf[i] = '\0';
   return buf;
 }
+
+char *
+sbrk(int n) {
+  return sys_sbrk(n, SBRK_EAGER);
+}
+
+char *
+sbrklazy(int n) {
+  return sys_sbrk(n, SBRK_LAZY);
+}
```

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index e154cee..419e727 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
......
@@ -39,12 +40,27 @@ uint64
 sys_sbrk(void)
 {
   uint64 addr;
+  int t;
   int n;

   argint(0, &n);
+  argint(1, &t);
......
```

sbrk 和 sbrklazy

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index e154cee..419e727 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
.....
 uint64
 sys_exit(void)
@@ -39,12 +40,27 @@ uint64
 sys_sbrk(void)
 {
   uint64 addr;
+  int t;
   int n;

   argint(0, &n);
+  argint(1, &t);
   addr = myproc()->sz;
-  if(growproc(n) < 0)
-    return -1;
+
+  if(t == SBRK_EAGER || n < 0) {
+    if(growproc(n) < 0) {
+      return -1;
+    }
+  } else {
+    // Lazily allocate memory for this process: increase its memory
+    // size but don't allocate memory. If the processes uses the
+    // memory, vmfault() will allocate it.
+    if(addr + n < addr)
+      return -1;
+    if(addr + n > TRAPFRAME)
+      return -1;
+    myproc()->sz += n;
+  }
   return addr;
 }
```

sbrk 和 sbrklazy

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index e154cee..419e727 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
.....
 uint64
 sys_exit(void)
@@ -39,12 +40,27 @@ uint64
 sys_sbrk(void)
 {
   uint64 addr;
+  int t;
   int n;

   argint(0, &n);
+  argint(1, &t);
   addr = myproc()->sz;
-  if(growproc(n) < 0)
-    return -1;
+
+  if(t == SBRK_EAGER || n < 0) {
+    if(growproc(n) < 0) {
+      return -1;
+    }
+  } else {
+    // Lazily allocate memory for this process: increase its memory
+    // size but don't allocate memory. If the processes uses the
+    // memory, vmfault() will allocate it.
+    if(addr + n < addr)
+      return -1;
+    if(addr + n > TRAPFRAME)
+      return -1;
+    myproc()->sz += n;
+  }
   return addr;
 }
```

```
diff --git a/kernel/trap.c b/kernel/trap.c
index ab346cc..95b8682 100644
--- a/kernel/trap.c
+++ b/kernel/trap.c
@@ -68,6 +68,9 @@ usertrap(void)
     syscall();
   } else if((which_dev = devintr()) != 0){
     // ok
+  } else if((r_scause() == 15 || r_scause() == 13) &&
+            vmfault(p->pagetable, r_stval(), (r_scause() == 13)? 1 : 0) != 0) {
+    // page fault on lazily-allocated page
   } else {
     printf("usertrap(): unexpected scause 0x%lx pid=%d\n", r_scause(), p->pid);
     printf("            sepc=0x%lx stval=0x%lx\n", r_sepc(), r_stval());
```

Interrupt	Exception Code	Description
0	0	Instruction address misaligned
0	1	Instruction access fault
0	2	Illegal instruction
0	3	Breakpoint
0	4	Load address misaligned
0	5	Load access fault
0	6	Store/AMO address misaligned
0	7	Store/AMO access fault
0	8	Environment call from U-mode
0	9	Environment call from S-mode
0	10-11	*Reserved*
0	12	Instruction page fault
0	13	Load page fault
0	14	*Reserved*
0	15	Store/AMO page fault
0	16-17	*Reserved*
0	18	Software check
0	19	Hardware error
0	20-23	*Reserved*
0	24-31	*Designated for custom use*
	32-47	*Reserved*
	48-63	*Designated for custom use*
	≥64	*Reserved*

sbrk 和 sbrklazy

```
diff --git a/kernel/sysproc.c b/kernel/sysproc.c
index e154cee..419e727 100644
--- a/kernel/sysproc.c
+++ b/kernel/sysproc.c
......
 uint64
 sys_exit(void)
@@ -39,12 +40,27 @@ uint64
 sys_sbrk(void)
 {
   uint64 addr;
+  int t;
   int n;

   argint(0, &n);
+  argint(1, &t);
   addr = myproc()->sz;
-  if(growproc(n) < 0)
-    return -1;
+
+  if(t == SBRK_EAGER || n < 0) {
+    if(growproc(n) < 0) {
+      return -1;
+    }
+  } else {
+    // Lazily allocate memory for this process: increase its memory
+    // size but don't allocate memory. If the processes uses the
+    // memory, vmfault() will allocate it.
+    if(addr + n < addr)
+      return -1;
+    if(addr + n > TRAPFRAME)
+      return -1;
+    myproc()->sz += n;
+  }
   return addr;
 }
```

```
diff --git a/kernel/trap.c b/kernel/trap.c
index ab346cc..95b8682 100644
--- a/kernel/trap.c
+++ b/kernel/trap.c
@@ -68,6 +68,9 @@ usertrap(void)
     syscall();
   } else if((which_dev = devintr()) != 0){
     // ok
+  } else if((r_scause() == 15 || r_scause() == 13) &&
+            vmfault(p->pagetable, r_stval(), (r_scause() == 13)? 1 : 0) != 0) {
+    // page fault on lazily-allocated page
   } else {
     printf("usertrap(): unexpected scause %p pid=%d\n", r_scause(), p->pid);
     printf("            sepc=%p stval=%p\n", r_sepc(), r_stval());
```

```
uint64 vmfault(pagetable_t pagetable, uint64 va, int read)
{
  uint64 mem;
  struct proc *p = myproc();

  if (va >= p->sz)
    return 0;
  va = PGROUNDDOWN(va);
  if(ismapped(pagetable, va)) {
    return 0;
  }
  mem = (uint64) kalloc();
  if(mem == 0)
    return 0;
  memset((void *) mem, 0, PGSIZE);
  if (mappages(p->pagetable, va, PGSIZE, mem, PTE_W|PTE_U|PTE_R) != 0) {
    kfree((void *)mem);
    return 0;
  }
  return mem;
}
```

sbrk 和 sbrklazy

```
diff --git a/kernel/vm.c b/kernel/vm.c
index d41500f..9548827 100644
--- a/kernel/vm.c
+++ b/kernel/vm.c
......
 void
 uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
@@ -196,12 +198,10 @@ uvmunmap(pagetable_t pagetable, uint64 va, uint64 npages, int do_free)
     panic("uvmunmap: not aligned");

   for(a = va; a < va + npages*PGSIZE; a += PGSIZE){
-    if((pte = walk(pagetable, a, 0)) == 0)
-      panic("uvmunmap: walk");
-    if((*pte & PTE_V) == 0)
-      panic("uvmunmap: not mapped");
-    if(PTE_FLAGS(*pte) == PTE_V)
-      panic("uvmunmap: not a leaf");
+    if((pte = walk(pagetable, a, 0)) == 0) // leaf page table entry allocated?
+      continue;
+    if((*pte & PTE_V) == 0)  // has physical page been allocated?
+      continue;
     if(do_free){
       uint64 pa = PTE2PA(*pte);
       kfree((void*)pa);
```

```
diff --git a/kernel/vm.c b/kernel/vm.c
index d41500f..9548827 100644
--- a/kernel/vm.c
+++ b/kernel/vm.c
......
@@ -302,9 +302,9 @@ uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)

   for(i = 0; i < sz; i += PGSIZE){
     if((pte = walk(old, i, 0)) == 0)
-      panic("uvmcopy: pte should exist");
+      continue;   // page table entry hasn't been allocated
     if((*pte & PTE_V) == 0)
-      panic("uvmcopy: page not present");
+      continue;   // physical page hasn't been allocated
     pa = PTE2PA(*pte);
     flags = PTE_FLAGS(*pte);
     if((mem = kalloc()) == 0)
```

```
diff --git a/kernel/vm.c b/kernel/vm.c
index d41500f..9548827 100644
--- a/kernel/vm.c
+++ b/kernel/vm.c
......
@@ -351,7 +351,9 @@ copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)

     pa0 = walkaddr(pagetable, va0);
     if(pa0 == 0) {
-      return -1;
+      if((pa0 = vmfault(pagetable, va0, 0)) == 0) {
+        return -1;
+      }
     }

     pte = walk(pagetable, va0, 0);
@@ -382,8 +384,11 @@ copyin(pagetable_t pagetable, char *dst, uint64 srcva, uint64 len)
   while(len > 0){
     va0 = PGROUNDDOWN(srcva);
     pa0 = walkaddr(pagetable, va0);
-    if(pa0 == 0)
-      return -1;
+    if(pa0 == 0) {
+      if((pa0 = vmfault(pagetable, va0, 0)) == 0) {
+        return -1;
+      }
+    }
     n = PGSIZE - (srcva - va0);
     if(n > len)
       n = len;
```

一些小实验

```c
#define BUF_SIZE (64 * 1024)
void cmd_memtest(void)
{
  uint64 t1, t2;
  char *buf;

  printf("memtest: start ......\n");
  t1 = uptime();
  for(int i = 0; i < 2000; i++) {
    buf = malloc(BUF_SIZE);
    if(buf == 0) {
      printf("Failed to allocate memory\n");
      exit(1);
    }

  }
  t2 = uptime();
  printf("memtest: end. Took %ld ticks\n", t2-t1);

  // To release memory allocated above.
  exit(0);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ memtest
memtest: start ......
memtest: end. Took 4 ticks
init: starting sh
$
```

一些小实验

```c
#define BUF_SIZE (64 * 1024)
void cmd_memtest(void)
{
  uint64 t1, t2;
  char *buf;

  printf("memtest: start ......\n");
  t1 = uptime();
  for(int i = 0; i < 2000; i++) {
    buf = malloc(BUF_SIZE);
    if(buf == 0) {
      printf("Failed to allocate memory\n");
      exit(1);
    }

  }
  t2 = up
  printf(

  // To r
  exit(0)
}
```

```
diff --git a/user/umalloc.c b/user/umalloc.c
index 4f753d7..c6e09c3 100644
--- a/user/umalloc.c
+++ b/user/umalloc.c
@@ -50,7 +50,7 @@ morecore(uint nu)

   if(nu < 4096)
     nu = 4096;
-  p = sbrk(nu * sizeof(Header));
+  p = sbrklazy(nu * sizeof(Header));
   if(p == SBRK_ERROR)
     return 0;
   hp = (Header*)p;
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ memtest
memtest: start ......
memtest: end. Took 4 ticks
init: starting sh
$
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ memtest
memtest: start ......
memtest: end. Took 1 ticks
init: starting sh
$
```

一些小实验

```
#define BUF_SIZE (64 * 1024)
void cmd_memtest(void)
{
  uint64 t1, t2;
  char *buf;

  printf("memtest: start ......\n");
  t1 = uptime();
  for(int i = 0; i < 2000; i++) {
    buf = malloc(BUF_SIZE);
    if(buf == 0) {
      printf("Failed to allocate memory\n");
      exit(1);
    }
    memset(buf, 0, BUF_SIZE); // Touch the memory to ensure it's allocated
  }
  t2 = uptime();
  printf("memtest: end. Took %ld ticks\n", t2-t1);

  // To release memory allocated above.
  exit(0);
}
```

一些小实验

```c
#define BUF_SIZE (64 * 1024)
void cmd_memtest(void)
{
  uint64 t1, t2;
  char *buf;

  printf("memtest: start ......\n");
  t1 = uptime();
  for(int i = 0; i < 2000; i++) {
    buf = malloc(BUF_SIZE);
    if(buf == 0) {
      printf("Failed to allocate memory\n");
      exit(1);
    }
    memset(buf, 0, BUF_SIZE); // Touch the memory to ensure it's allocated
  }
  t2 = uptime();
  printf("memtest: end. Took %ld ticks\n", t2-t1);

  // To release memory allocated above.
  exit(0);
}
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ memtest
memtest: start ......
memtest: end. Took 6 ticks
init: starting sh
$
```

一些小实验

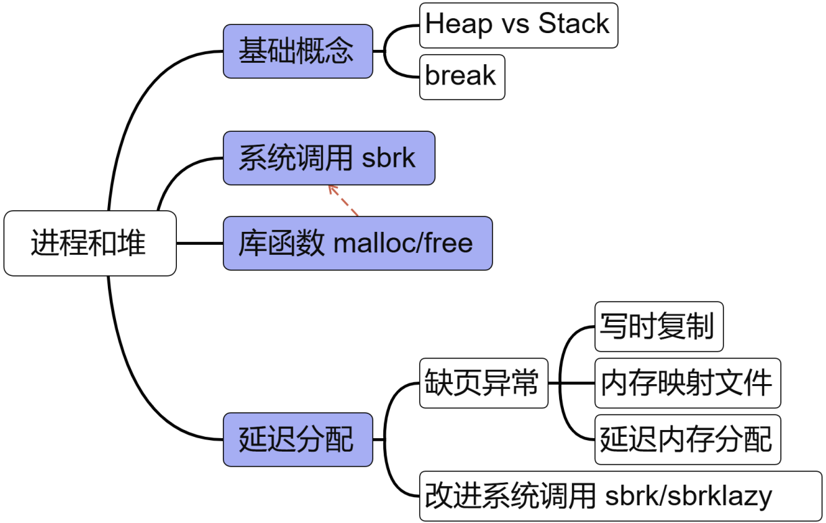
```
#define BUF_SIZE (64 * 1024)
void cmd_memtest(void)
{
  uint64 t1, t2;
  char *buf;

  printf("memtest: start ......\n");
  t1 = uptime();
  for(int i = 0; i < 2000; i++) {
    buf = malloc(BUF_SIZE);
    if(buf == 0) {
      printf("Failed to allocate memory\n");
      exit(1);
    }
    memset(buf, 0, BUF_SIZE); // Touch the memory to ensure it's allocated
  }
  t2 = up
  printf(

  // To r
  exit(0)
}
```

```
diff --git a/user/umalloc.c b/user/umalloc.c
index 4f753d7..c6e09c3 100644
--- a/user/umalloc.c
+++ b/user/umalloc.c
@@ -50,7 +50,7 @@ morecore(uint nu)

   if(nu < 4096)
     nu = 4096;
-  p = sbrk(nu * sizeof(Header));
+  p = sbrklazy(nu * sizeof(Header));
   if(p == SBRK_ERROR)
     return 0;
   hp = (Header*)p;
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ memtest
memtest: start ......
memtest: end. Took 6 ticks
init: starting sh
$
```

```
$ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m
128M -smp 3 -nographic

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ memtest
memtest: start ......
memtest: end. Took 12 ticks
init: starting sh
$
```

本章总结


进程和堆
基础概念
Heap vs Stack
break
系统调用 sbrk
库函数 malloc/free
延迟分配
缺页异常
写时复制
内存映射文件
延迟内存分配
改进系统调用 sbrk/sbrklazy

谢谢